

Momentum, Preconditioning, and Beyond: Practical Advances in Optimization

Vienna Deep Learning Meetup #73

Ionut-Vlad Modoranu

May 19, 2026

Agenda

- Chapter 1: Intro to Optimization
 - The Basics
 - SGD & Momentum
 - Loss Landscape Geometry
- Chapter 2: Preconditioning
 - Diagonal:
 - AdaGrad & Adam
 - MicroAdam (ours) - **postponed!**
 - Non-Diagonal:
 - Shampoo & DASH (ours)
 - Muon, Dion & Trion (ours)

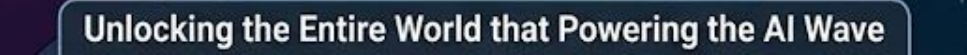
The View from the Top: Two Lines of PyTorch Code

```
1 # start part of training
2 optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
3 optimizer.zero_grad()
4 optimizer.load()
5
6 for batch in ranger(batches):
7     loss.backward()
8     optimizer.step()
9     data = loss.oam_end(dsatis, batches))
```

Behind the Closed Door: The Engine of AI Optimization

The collage illustrates the components of AI optimization:

- Autograd** and **Automatic Differentiation**: Represented by a diagram of a neural network with arrows indicating the flow of gradients.
- Closed/Black Box**: Represented by a safe with a sign that says "Closed/Black Box".
- Gradient Descent landscapes**: Represented by a 3D surface plot of a loss function $L = (h_t)$ with a gradient vector $g_t = (r_t - y_t)$ and the update rule $h_{t+1} = \frac{\eta}{\sqrt{\phi_t + \epsilon}}$.
- Backpropagation**: Represented by a small diagram of a neural network with the update rule $\phi_t = \phi + \epsilon$.



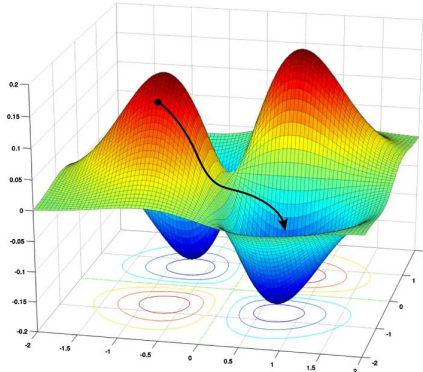
Chapter 1

Intro to Optimization

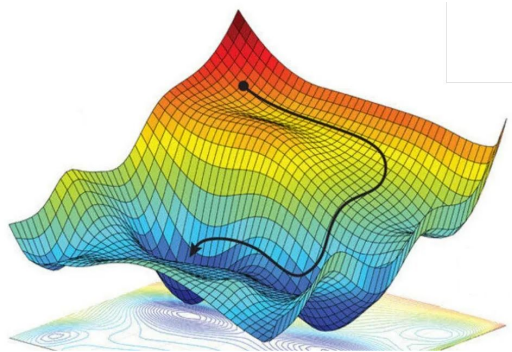


1. Intro to Optimization: The Basics

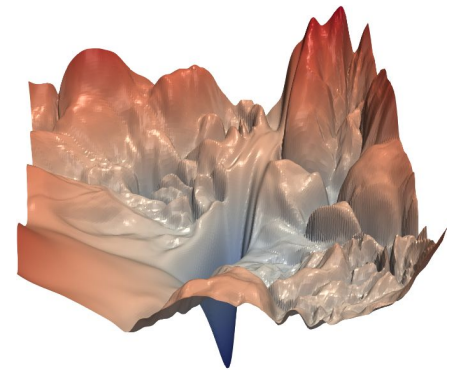
- Find the parameters of a model minimizing an error function
 - Landscape created by model and data
- Goal
 - Minimize number of steps to reach optimum (minimum)



Taken from [1]



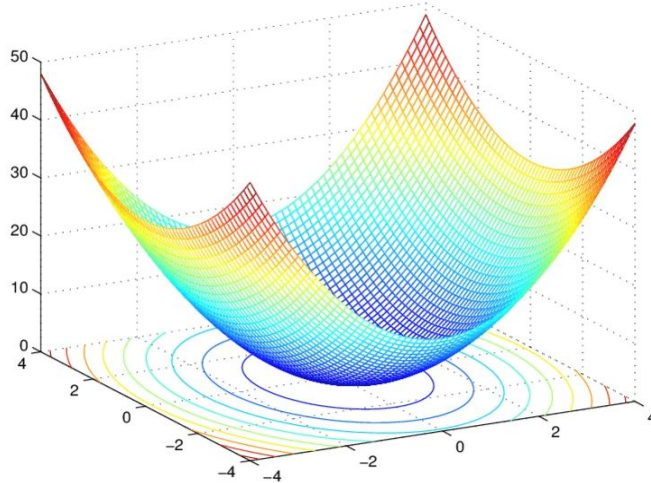
Taken from [2]



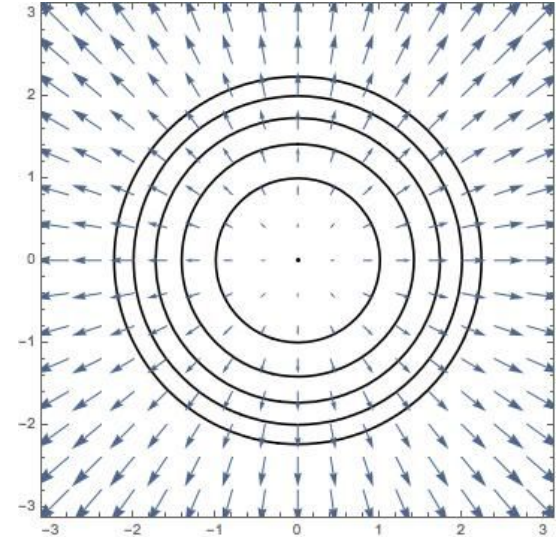
Taken from [3]

1. Intro to Optimization: The Basics

- Gradient-based optimization:
 - how much function increases in one particular direction
 - gradient is perpendicular to level sets
 - use negative gradient as descent direction



Taken from [4]



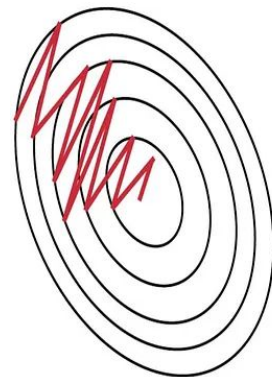
Taken from [5]

1. Intro to Optimization: SGD & Momentum

- **Stochastic Gradient Descent:**

- hiker taking independent steps
- no memory of the past
- small progress in certain directions

$$\theta_{t+1} = \theta_t - \eta \cdot g_t$$



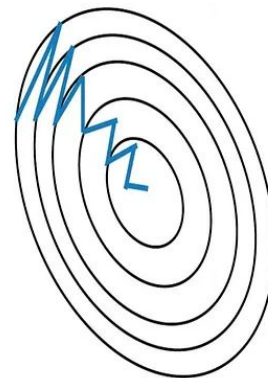
Taken from [2]

- **Momentum:**

- heavy ball rolling down the hill
- forgets old gradients
- gain velocity in consistent directions
- cancel out oscillating directions

$$m_t = \beta m_{t-1} + (1 - \beta) g_t$$

$$\theta_{t+1} = \theta_t - \eta \cdot m_t$$



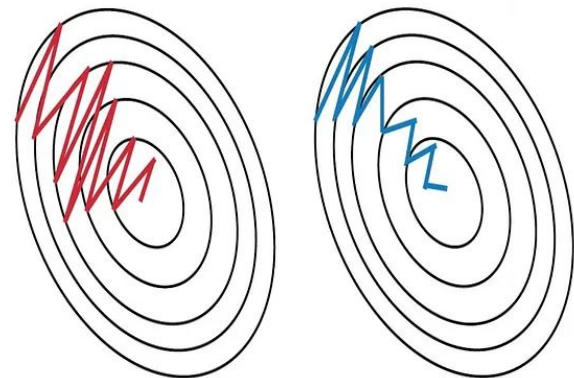
1. Intro to Optimization: Loss Landscape Geometry

- Shape of level sets:

- circle: desired
- ellipses: oscillations in the trajectory (zig-zags) - slow

- Curvature at a specific point (x, y):

- characterized by the Hessian matrix $\mathbf{H}$ at (x, y):
- condition number: quantifies the difficulty of landscape
- what to do when κ_H is large?



SGD

SGD + Momentum

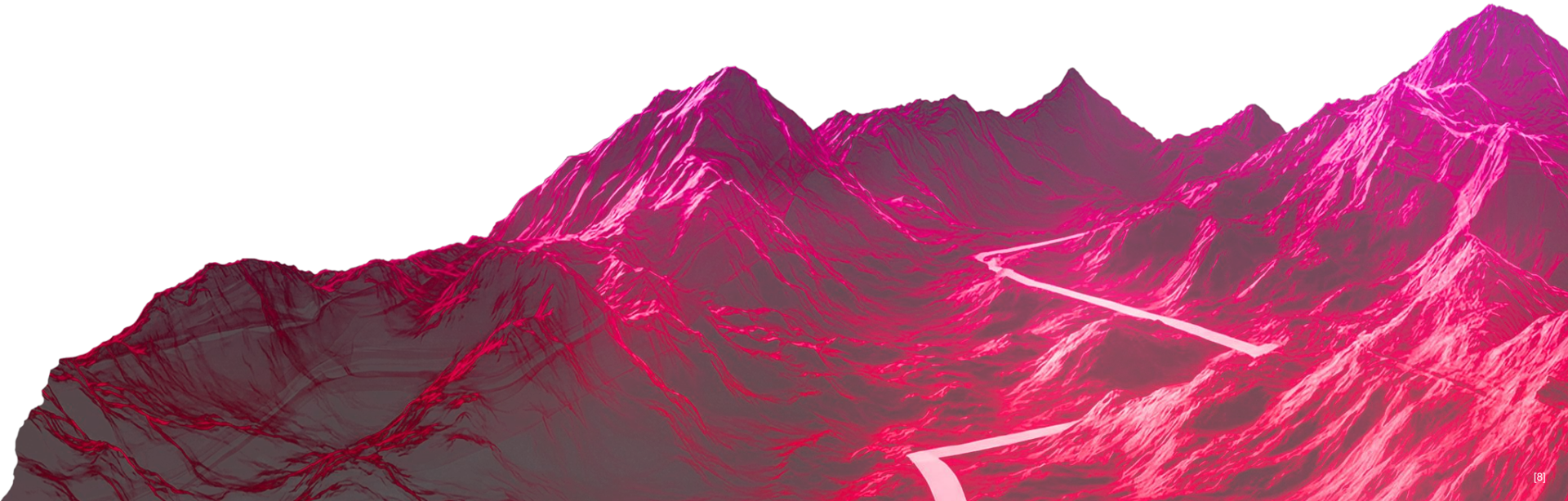
$$H_{ij} = \frac{\partial L}{\partial \theta_i \partial \theta_j}$$

$$\kappa_H = \frac{\lambda_{\max}}{\lambda_{\min}}$$

Preconditioning!

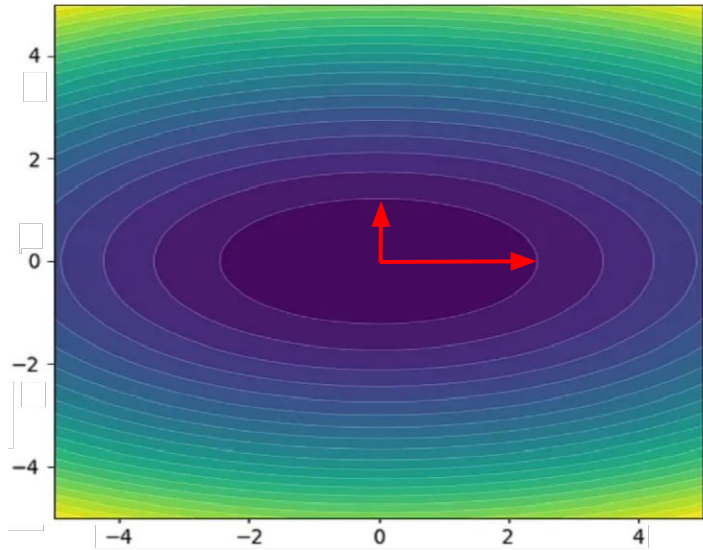
Chapter 2

Preconditioning



2. Preconditioning

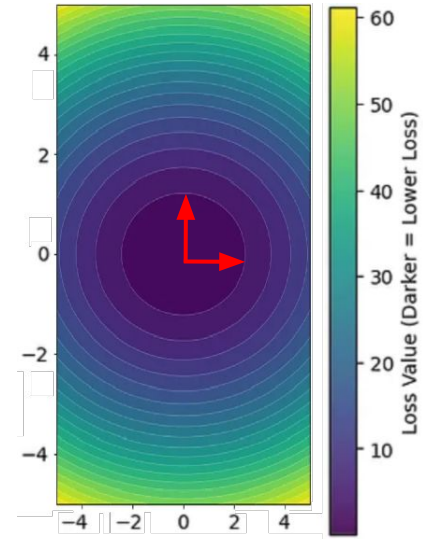
- Space rescaling:
 - Apply a transformation to the space
 - Convert level sets: ellipses $\rightarrow$ circles (ideally)



$$\lambda_1 = 20 \quad \lambda_2 = 4 \quad \kappa = 5$$

Preconditioning:
 $y = S \cdot x$

How to choose S ?
Inverse Hessian!



$$\lambda_1 = 1 \quad \lambda_2 = 1 \quad \kappa = 1$$

2. Preconditioning

- Ideal optimization algorithm: Newton's method

- Impossible to compute/store for Deep Learning
- Expensive: compute & invert at each step t

$$\theta_{t+1} = \theta_t - \eta H_t^{-1} g_t$$

- Cheaper replacements to H :

- Avoid 2 back-props
- Gradient covariance as “second order” proxy
- AdaGrad^[14] optimizer:
 - sum up all past outer product
 - slow convergence
 - use diagonal as a cheaper version

$$H_t \approx g_t g_t^\top$$

$$G_t = \sum_{i=1}^t g_i g_i^\top = G_{t-1} + g_t g_t^\top$$

$$\theta_{t+1} = \theta_t - \eta G_t^{-1/2} g_t$$

$$\theta_{t+1} = \theta_t - \eta \text{diag}(G_t^{-1/2}) g_t$$

$$\theta_{t+1} = \theta_t - \eta \text{diag}(\epsilon I + G_t^{1/2})^{-1} g_t$$

2. Preconditioning: Adam Optimizer

- Different versions of AdaGrad:
 - add momentum to $\mathbf{G}_t$ and call it $\mathbf{v}_t$
 - obtain Geoffrey Hinton's **RMSProp**
 - use a sliding window due to momentum

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\epsilon + \sqrt{v_t}}$$

- Diagonal preconditioner: Adam^[13]
 - add momentum to $\mathbf{g}_t$ as $\mathbf{m}_t$
 - obtain the final Adam expression
 - **Adam is SGD with adjusted element-wise learning-rate!**

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\epsilon + \sqrt{v_t}}$$

$$= \theta_t - \frac{\eta}{\epsilon + \sqrt{v_t}} m_t$$

2. Preconditioning: Non-Diagonal Optimizers (Shampoo)

- Shampoo^[11,12] Optimizer:
 - Takes into consideration the 2D structure of gradient $\mathbf{G}_t$
 - Uses row-wise and column-wise covariances $\mathbf{L}_t$ and $\mathbf{R}_t$
 - Computes $-1/4$ power of $\mathbf{L}_t$ and $\mathbf{R}_t$ via **Eigen Value Decomposition (EVD)**

$$\mathbf{G}_t = \nabla_{\theta} f(\theta_t)$$

$$\mathbf{L}_t = \beta \mathbf{L}_{t-1} + (1 - \beta) \mathbf{G}_t \mathbf{G}_t^{\top}$$

$$\mathbf{R}_t = \beta \mathbf{R}_{t-1} + (1 - \beta) \mathbf{G}_t^{\top} \mathbf{G}_t$$

$$\theta_{t+1} = \theta_t - \eta \mathbf{L}_t^{-1/4} \mathbf{G}_t \mathbf{R}_t^{-1/4}$$

2. Preconditioning: Non-Diagonal Optimizers (Shampoo)

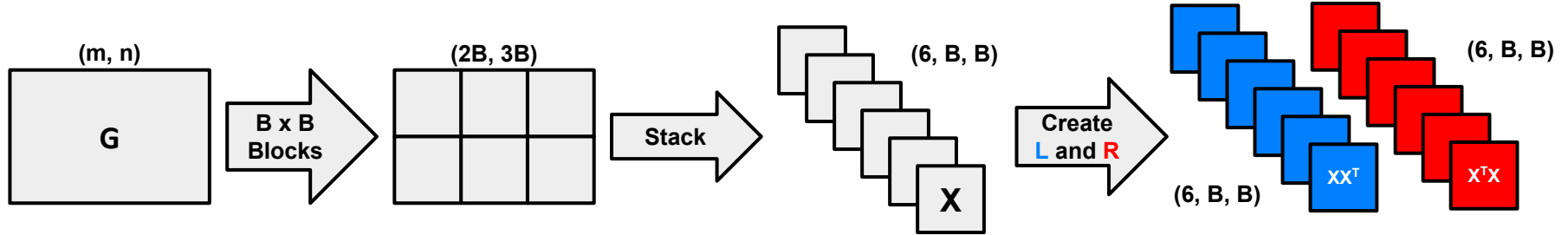
- Pros:
 - won ML Commons AlgoPerf competition
 - produces lower activation outliers → models trained with Shampoo are easier to compress
- Cons:
 - EVD is slow
 - Heuristic: split $\mathbf{G}_t$ into blocks of size $\mathbf{B}$ for preconditioning
 - Coupled-Newton is numerically unstable
 - iterative procedure for $-1/4$ power via matrix multiplications
 - default to EVD
 - Distributed Shampoo from META: overengineered & jack of all trades, master of none
 - framework for multiple optimizers
 - efficiency is shadowed by features

2. Preconditioning: Non-Diagonal Optimizers (DASH)

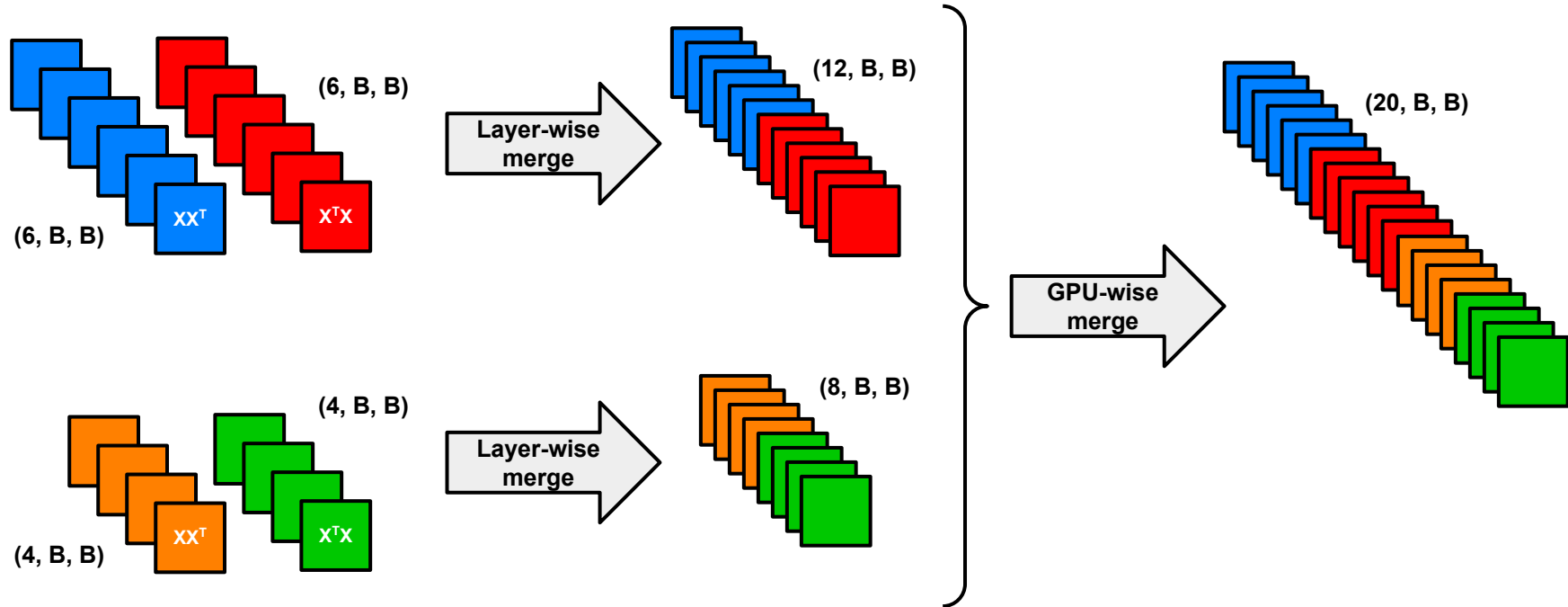
- Our Contribution: **DASH** (**D**istributed **A**ccelerated **S**Hampoo)
 - inherit the heuristic: split gradient into blocks of size **B**
 - compute $\mathbf{L}_t$ and $\mathbf{R}_t$ block-wise
- Key observation:
 - stack all blocks from $\mathbf{L}_t$ and $\mathbf{R}_t$ into a 3D structure merged at GPU level
 - EVD works for 3D matrices (batches)
 - Suitable for iterative procedures that estimate $-1/4$ power of a matrix
 - matmul-based
 - ideal for GPUs

Let's visualize this process!

2. Preconditioning: Non-Diagonal Optimizers (DASH)



2. Preconditioning: Non-Diagonal Optimizers (DASH)



2. Preconditioning: Non-Diagonal Optimizers (DASH)

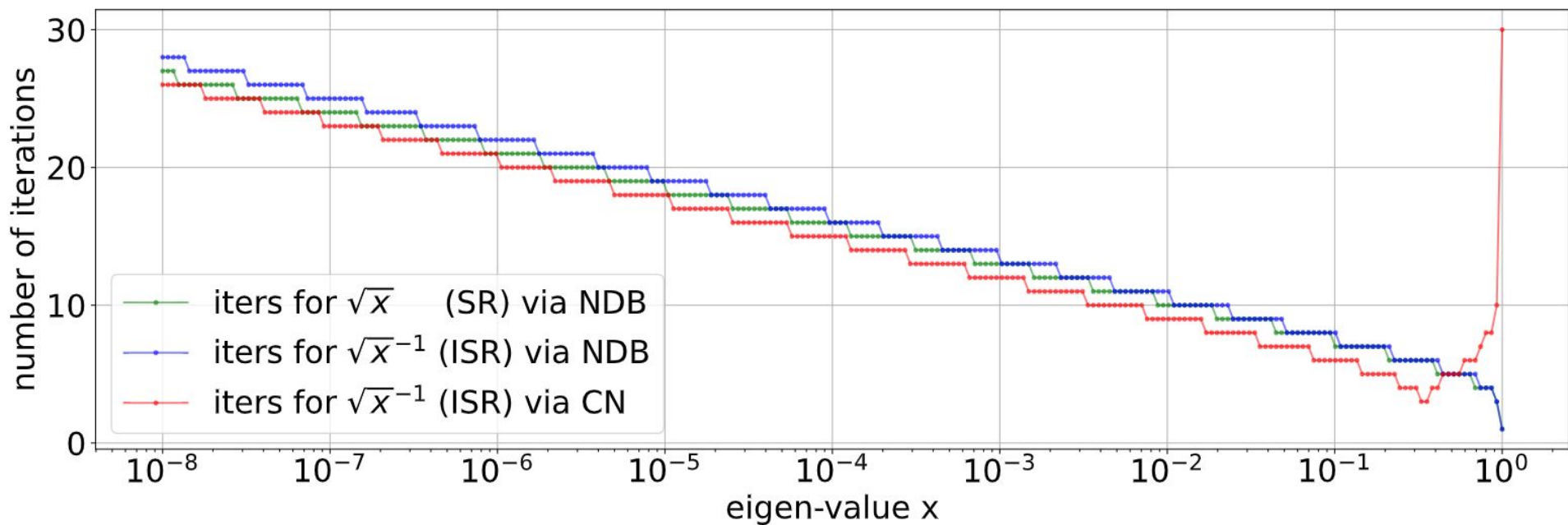
- Our solution for $-1/4$ power: Newton-DB
 - Already exists in Numerical Linear Algebra (never used in the field so far)
 - An iterative procedure based only on matmuls
 - Given $\mathbf{A} \in \mathbf{R}^{n \times n}$, create $\mathbf{Y}_k \rightarrow \mathbf{A}^{1/2}$ and $\mathbf{Z}_k \rightarrow \mathbf{A}^{-1/2}$
 - For $-1/4$, we need to compute $\mathbf{K} = \mathbf{A}^{1/2}$ followed by another call for $\mathbf{K}^{-1/2}$ which is $\mathbf{A}^{-1/4}$

$$\begin{aligned}
 Y_0 &= A, & Z_0 &= I_n \\
 E_{k-1} &= \frac{1}{2}(3I - Z_{k-1}Y_{k-1}) \\
 Y_k &= Y_{k-1}E_{k-1}, & Z_k &= E_{k-1}Z_{k-1}
 \end{aligned}$$

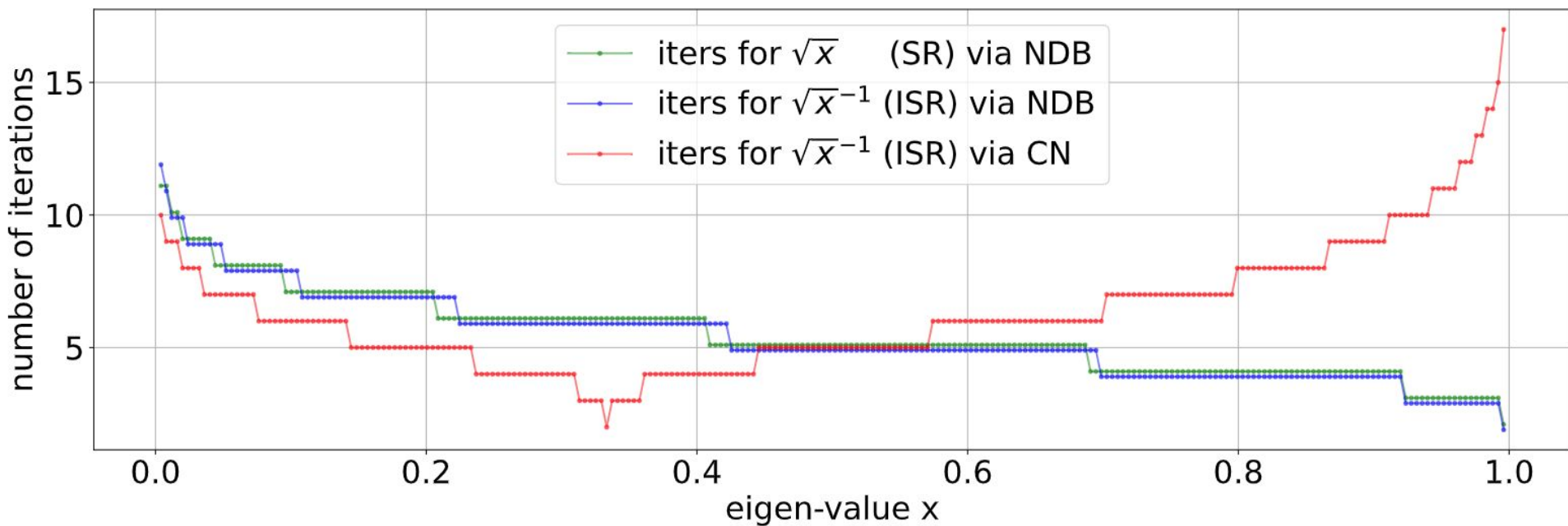
2. Preconditioning: Non-Diagonal Optimizers (DASH)

- Convergence criteria: $\lambda_{\max} < 1$
 - we need to scale the matrix to satisfy the condition
 - **Option 1:** scale matrix by Frobenius norm: $\lambda_{\max}(\mathbf{A}) \leq \|\mathbf{A}\|_F$ (in practice, this gap is **10x**)
 - default in Coupled-Newton
 - **Option 2:** scale by λ_{PI} estimated via Power Iteration & Rayleigh Quotient
 - we use the 3D structure of shape (N, B, B)
 - increased efficiency due to batch matrix-vector (matvec) multiplication
 - scale by $2\lambda_{PI}$ because $\lambda_{PI} \leq \lambda_{\max}$
- Advantages of Option 2:
 - Resulting eigenvalues lie in **[0, 0.5]** instead of **[0, 0.1]** (for Frobenius in Option 1)
 - Lower number of iterations to converge for NewtonDB (see next)

2. Preconditioning: Non-Diagonal Optimizers (DASH)



2. Preconditioning: Non-Diagonal Optimizers (DASH)



2. Preconditioning: Non-Diagonal Optimizers (DASH)

Method	Frequency	Val PPL		Runtime (ms)		Normaliza tion
		DIST	DASH	DIST	DASH	
EVD	1	11.80	11.81	3080	2850	-
	10	11.91	11.92	355	313	-
Coupled Newton	1	11.87	11.87	666	136	FRO / FP32
		11.87	11.87	471	119	FRO / FP16
Newton DB	1	11.76	11.77	558	174	FRO / FP32
		11.69	11.68	740	177	PI / FP32

2. Preconditioning: Non-Diagonal Optimizers (DASH)

Method	Frequency	Val PPL		Runtime (ms)		Normaliza tion
		DIST	DASH	DIST	DASH	
EVD	1	11.80	11.81	3080	2850	-
	10	11.91	11.92	355	313	-
Coupled Newton	1	11.87	11.87	666	136	FRO / FP32
		11.87	11.87	471	119	FRO / FP16
Newton DB	1	11.76	11.77	558	174	FRO / FP32
		11.69	11.68	740	177	PI / FP32

2. Preconditioning: Non-Diagonal Optimizers (DASH)

Method	Frequency	Val PPL		Runtime (ms)		Normaliza tion
		DIST	DASH	DIST	DASH	
EVD	1	11.80	11.81	3080	2850	-
	10	11.91	11.92	355	313	-
Coupled Newton	1	11.87	11.87	666	136	FRO / FP32
		11.87	11.87	471	119	FRO / FP16
Newton DB	1	11.76	11.77	558	174	FRO / FP32
		11.69	11.68	740	177	PI / FP32

2. Preconditioning: Non-Diagonal Optimizers (DASH)

Method	Frequency	Val PPL		Runtime (ms)		Normaliza tion
		DIST	DASH	DIST	DASH	
EVD	1	11.80	11.81	3080	2850	-
	10	11.91	11.92	355	313	-
Coupled Newton	1	11.87	11.87	666	136	FRO / FP32
		11.87	11.87	471	119	FRO / FP16
Newton DB	1	11.76	11.77	558	174	FRO / FP32
		11.69	11.68	740	177	PI / FP32

2. Preconditioning: Non-Diagonal Optimizers (DASH)

Method	Frequency	Val PPL		Runtime (ms)		Normaliza tion
		DIST	DASH	DIST	DASH	
EVD	1	11.80	11.81	3080	2850	-
	10	11.91	11.92	355	313	-
Coupled Newton	1	11.87	11.87	666	136	FRO / FP32
		11.87	11.87	471	119	FRO / FP16
Newton DB	1	11.76	11.77	558	174	FRO / FP32
		11.69	11.68	740	177	PI / FP32

2. Preconditioning: Non-Diagonal Optimizers (DASH)

Method	Frequency	Val PPL		Runtime (ms)		Normaliza tion
		DIST	DASH	DIST	DASH	
EVD	1	11.80	11.81	3080	2850	-
	10	11.91	11.92	355	313	-
Coupled Newton	1	11.87	11.87	666	136	FRO / FP32
		11.87	11.87	471	119	FRO / FP16
Newton DB	1	11.76	11.77	558	174	FRO / FP32
		11.69	11.68	740	177	PI / FP32

2. Preconditioning: Non-Diagonal Optimizers (DASH)

Method	Frequency	Val PPL		Runtime (ms)		Normaliza tion
		DIST	DASH	DIST	DASH	
EVD	1	11.80	11.81	3080	2850	-
	10	11.91	11.92	355	313	-
Coupled Newton	1	11.87	11.87	666	136	FRO / FP32
		11.87	11.87	471	119	FRO / FP16
Newton DB	1	11.76	11.77	558	174	FRO / FP32
		11.69	11.68	740	177	PI / FP32

5.60x

2. Preconditioning: Non-Diagonal Optimizers (DASH)

Method	Frequency	Val PPL		Runtime (ms)		Normaliza tion
		DIST	DASH	DIST	DASH	
EVD	1	11.80	11.81	3080	2850	-
	10	11.91	11.92	355	313	-
Coupled Newton	1	11.87	11.87	666	136	FRO / FP32
		11.87	11.87	471	119	FRO / FP16
Newton DB	1	11.76	11.77	558	174	FRO / FP32
		11.69	11.68	740	177	PI / FP32

3.76x

2. Preconditioning: Non-Diagonal Optimizers (Muon)

- Muon^[9]:

- SGD + orthogonalization
- All singular values are pushed towards 1:
- Balanced contribution from all directions

$$\kappa_M = 1$$

- Orthogonalization:

- Optimal: via **S**ingular **V**alue **D**ecomposition (SVD) → expensive
- Approximation: via Newton Schulz (NS) → faster than SVD
 - apply odd polynomial applied 5 times to **M**:

$$M = U \Sigma V^\top$$

$$p(x) = ax + bx^3 + cx^5$$

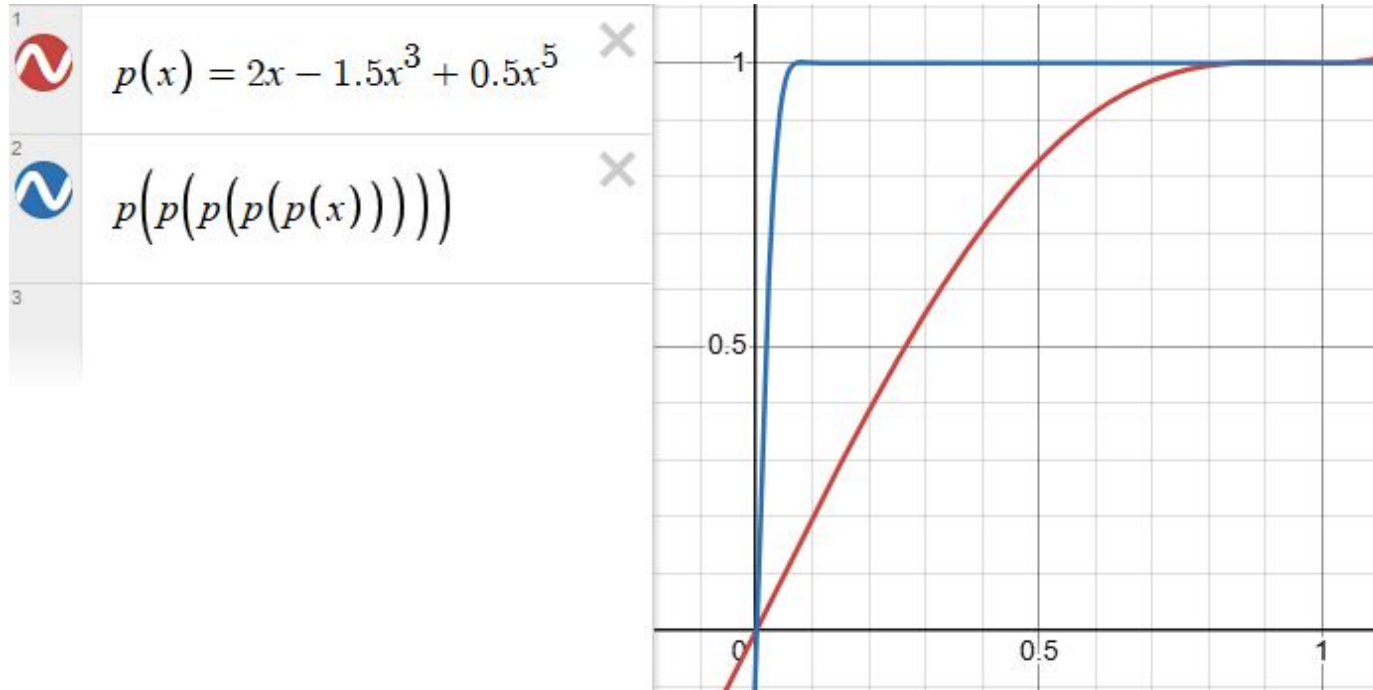
- **a, b, c** are chosen such that:

$$p^{(5)}(x) \approx 1$$

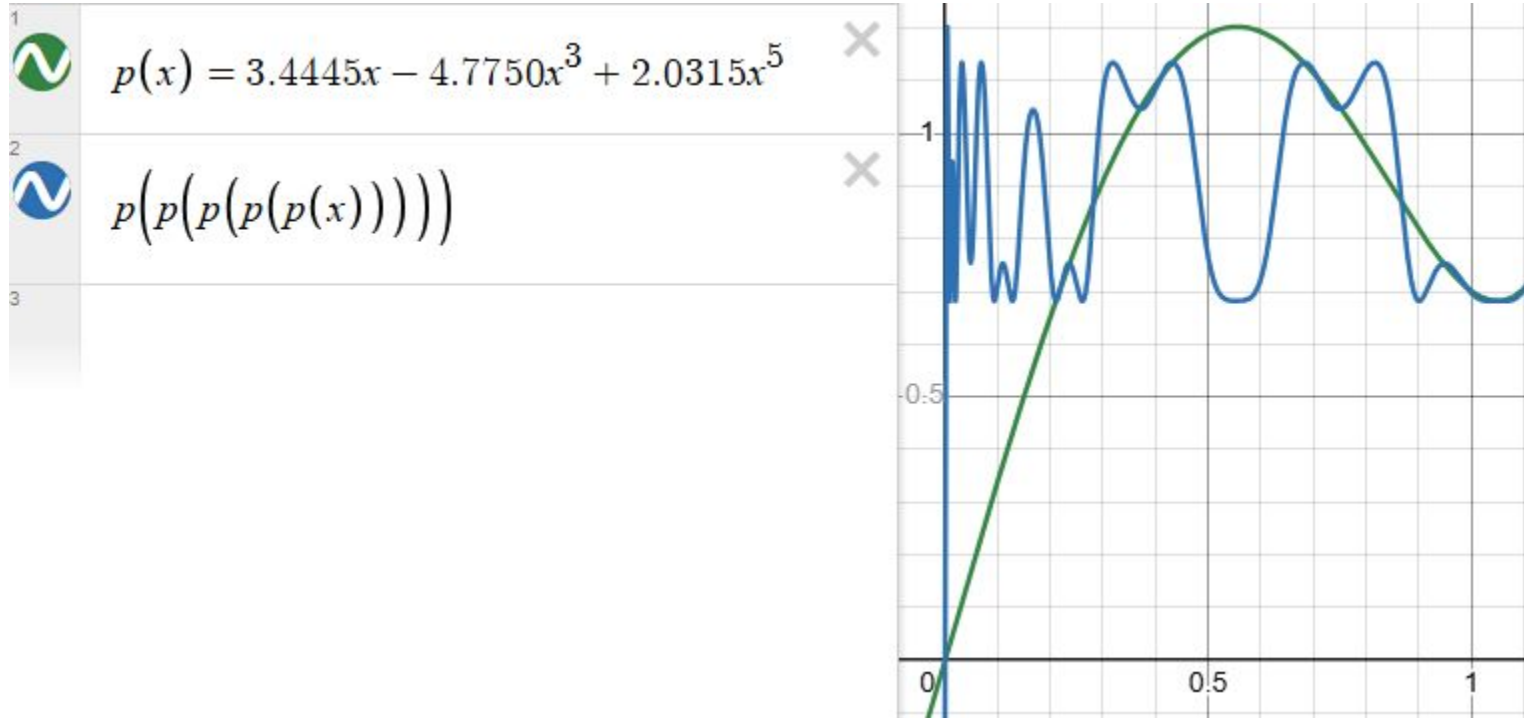
- **p(x)** applied to **M** acts on **Σ**:

$$p^{(5)}(M) = U p^{(5)}(\Sigma) V^\top$$

2. Preconditioning: Non-Diagonal Optimizers (Muon)



2. Preconditioning: Non-Diagonal Optimizers (Muon)



2. Preconditioning: Non-Diagonal Optimizers (Muon)

- Pros:
 - Fast learning: reaches a specific training loss faster than Adam
 - Does not store additional states: same memory usage as SGD
- Cons:
 - High FLOPs usage:
 - 15 matmuls for Newton-Schulz
 - Communication-intensive between GPUs in distributed setup
 - faster version: **Dion!**

2. Preconditioning: Non-Diagonal Optimizers (Dion)

- Dion^[10]: **D**istributed **O**rthonormalization

- Uses a low-rank version of momentum matrix $\mathbf{M}$:

$$M \approx PR^\top$$

$$\begin{aligned} M &\in \mathbb{R}^{m \times n} \\ P &\in \mathbb{R}^{m \times r} \\ R &\in \mathbb{R}^{n \times r} \end{aligned}$$

- Simplified Dion algorithm:

- Power iteration: computes the low-rank matrices $\mathbf{P}_t$ and $\mathbf{R}_t$
 - Orthogonalization of $\mathbf{P}$ involves QR-decomposition

$$M_t = \beta M_{t-1} + (1 - \beta) G_t$$

$$P_t, R_t = \text{POWERITERATION}(M_t)$$

$$\theta_{t+1} = \theta_t - \eta \cdot \text{QR}(P_t) \cdot R_t^\top$$

2. Preconditioning: Non-Diagonal Optimizers (Dion)

- Pros:
 - Reduced communication in distributed settings
 - sending low-rank matrices is cheaper
- Cons:
 - Power iteration:
 - iterative procedure, only one step is run $\rightarrow$ approximation, large reconstruction error
 - QR-decomposition:
 - Same class as SVD/EVD of expensive Numerical Linear Algebra routines
 - Slow on GPUs compared to matmuls
 - Requires using $\mathbf{P}$ and $\mathbf{R}$ in addition to momentum $\mathbf{M} \rightarrow$ increased memory usage over Muon

How can we improve runtime and model performance?

2. Preconditioning: Non-Diagonal Optimizers (Trion)

- Optimal procedure to compute the subspace:
 - SVD is expensive, Power iteration is inaccurate
- Our approach: Dynamic Column Selection Algorithm
 - given orthogonal $\mathbf{Q}$, compute $\mathbf{S} = \mathbf{G} \mathbf{Q}$
 - rank columns in $\mathbf{S}$ based on $\mathbf{L}_1$ -norm
 - select a subset of size r with largest norm
 - select r columns from $\mathbf{Q}$ as $\mathbf{P} = \mathbf{Q}[:, \text{subset}]$
 - use the projection matrix $\mathbf{P}$ to move between subspaces

$$\begin{bmatrix} \text{---} G_1 \text{---} \\ \text{---} G_2 \text{---} \\ \vdots \\ \text{---} G_n \text{---} \end{bmatrix} \begin{bmatrix} | & | & \cdots & | \\ Q_1 & Q_2 & \cdots & Q_n \\ | & | & \cdots & | \end{bmatrix} = \begin{bmatrix} G_1^\top Q_1 & G_1^\top Q_2 & \cdots & G_1^\top Q_n \\ G_2^\top Q_1 & G_2^\top Q_2 & \cdots & G_2^\top Q_n \\ \vdots & \vdots & \cdots & \vdots \\ G_n^\top Q_1 & G_n^\top Q_2 & \cdots & G_n^\top Q_n \end{bmatrix}$$

2. Preconditioning: Non-Diagonal Optimizers (Trion)

- Any orthogonal **Q** works → we choose DCT (JPEG)

$$Q_{ij} = \sqrt{2/n} \cdot \cos \frac{i(2j+1)\pi}{2n}$$

- Makhoul's algorithm for **S = G Q**:
 - O(n² log(n))** instead of **O(n³)** for matmul
 - faster for old GPUs with few/slow tensor-cores

Matrix Size	GPU	TCs	Matmul	FFT	Ratio	Faster
(4096, 4096)	H100	✓	0.36 ms	0.39 ms	0.92 ×	MM
		✗	2.67 ms	0.37 ms	7.08 ×	FFT
	RTX 3090	✓	4.00 ms	0.89 ms	4.48 ×	FFT
		✗	5.84 ms	0.91 ms	6.42 ×	FFT
	RTX 2080	✓	10.67 ms	1.33 ms	7.99 ×	FFT
		✗	10.81 ms	1.35 ms	7.95 ×	FFT

2. Preconditioning: Non-Diagonal Optimizers (Trion)

- Use the approximate Newton-Schulz method instead of QR:
 - Similarities: $\mathbf{S} = \text{Makhoul}(\mathbf{M})$
 - Ranking: $\text{idx} = \text{DynamicColumnSelection}(\mathbf{S})$
 - Projection matrix: $\mathbf{P} = \mathbf{Q}[:, \text{idx}]$
 - Low-rank momentum: $\mathbf{m} = \mathbf{S}[:, \text{idx}]$
 - Orthogonalization: $\mathbf{m}_{\text{ortho}} = \text{NewtonSchulz}(\mathbf{m})$
 - Update model: $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \mathbf{m}_{\text{ortho}} \mathbf{P}^T$

2. Preconditioning: Non-Diagonal Optimizers (Trion)

- Results for rank $r = 128$

Rank r	Optimizer	Runtime		Val PPL	
		800M	1.3B	800M	1.3B
128 (6.25%)	Dion	8h 24m	19h 42m	12.52	11.56
	Trion	8h 5m	19h 13m	12.25	11.27
2048 (full)	Muon	8h 25m	19h 17m	12.05	11.13

2. Preconditioning: Non-Diagonal Optimizers (Trion)

- Results for rank $r = 512$

Rank r	Optimizer	Runtime		Val PPL	
		800M	1.3B	800M	1.3B
512 (25%)	Dion	8h 48m	20h 44m	12.30	11.36
	Trion	8h 3m	19h 13m	12.18	11.25
2048 (full)	Muon	8h 25m	19h 17m	12.05	11.13

Questions?

Shoutout to my collaborators



Dan Alistarh



Mher Safaryan



Erik Schultheis



Philip Zmushko

References

- [1] [Medium article #1](#)
- [2] [Medium article #2](#)
- [3] Li et al., [Visualizing the loss landscape of neural nets](#), NeurIPS 2018
- [4] [Researchgate Article](#)
- [5] [Level Sets and Gradient Flow](#)
- [6] [Medium Article #3](#)
- [7] [TowardsAI post](#)
- [8] Chapter images from StrangeWorks: [Chapter 1](#), [Chapter 2](#)
- [9] [Muon optimizer](#)
- [10] Ahn et al., [Dion: Distributed Orthonormalized Updates](#), 2025
- [11] Shi et al., [A Distributed Data-Parallel PyTorch Implementation of the Distributed Shampoo Optimizer for Training Neural Networks At-Scale](#), 2023
- [12] Gupta et al., [Shampoo: Preconditioned Stochastic Tensor Optimization](#), 2018
- [13] Kingma et al., [Adam: A Method for Stochastic Optimization](#), 2014
- [14] Duchi et al., [Adaptive Subgradient Methods for Online Learning and Stochastic Optimization](#), JMLR 2014

Code & Papers

Trion



ICLR 2026 



DASH



ICML 2026 